



Creating Slides

Abdul Saboor¹, Unknown Author²

Sep 12, 2026

¹My University is somewhere in the middle of nowhere

²Their University is somewhere in the middle of nowhere

🔍 Looking for Slides Controls?

- Right-click (or click the footer) to open the context menu for settings and contents.
- Hover top-left and click ▶ to open the menu when the footer or right-click is unavailable.
- Click a footer section (when shown) to jump there quickly.

✓ Show Code



Contents

- 1. Introduction**
2. Adding informative TOC
3. Plotting and DataFrame
4. Simple Animations with Frames
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides



Introduction

To see how commands work, use `ipyslides.docs()` to see the documentation. Here we will focus on using some of that functionality to create slides.

Note

This slide was built purely from markdown, so you can create a variable `test` to overwrite this →

Exception: Could not resolve '%{test}':

NameError: name 'test' is not defined

You can update this variable by `'Slides[int,|list|slice].vars.update'` or by defining it in notebook if `'Auto Rebuild'` is enabled.

Version: 7.3.88

Contents

1. Introduction
2. **Adding informative TOC**
3. Plotting and DataFrame
4. Simple Animations with Frames
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides

This comprehensive TOC was created using inline `toc` function.
See `slides.xmd.syntax` for details.

- Item 1
- Item 2

$$E = mc^2$$



💡 Variables Update

Above `btn` variable can be updated later via
`Slides[number,].vars.update` method.

Markdown

```
1 [section! Adding informative TOC /]
2 ```columns 4 6
3 [toc! ### Contents .. True /]
4 --
5 This comprehensive TOC was created using inline `toc` function. See `slides.xmd.syntax` for details.
6
7 ::: columns.inline 1 1
8   - Item 1
9   - Item 2
10  --
11  $$ E = mc^2 $$
12
```



Plotting with Matplotlib



Python

```
1 slides.css(bg1 = 'linear-gradient(to right, #FFDAB9  
2 import numpy as np, matplotlib.pyplot as plt  
3 plt.rcParams['svg.fonttype'] = 'none' # Global setti  
4 x = np.linspace(0, 2*np.pi)  
5 with plt.style.context('ggplot'):  
6     fig, ax = plt.subplots(figsize=(3.4, 2.6))  
7     _ = ax.plot(x, np.cos(x))  
8 write(ax, s.focus([0, 3, 4]))  
9 write('Double click on the plot to focus on it!')
```

Double click on the plot to focus on it!



Writing Pandas DataFrame

Install pandas to view output

Python

```
1 try:
2     import ipywidgets as ipw
3     import plotly.graph_objects as go
4     fig = slides.patched_plotly(go.FigureWidget()) # prefere Widget for interactivity and correct display
5     fig.add_trace(go.Bar(y=[1,5,8,9], customdata=["A","B"]))
6
7     # We have clicked and selected traits on patched plotly
8     html = ipw.HTML()
9     def observe_click(change):
10         html.value = "<br/>".join(f" {k} = {v}" for k, v in change['new'].items())
11     fig.observe(observe_click, names='clicked')
12     box = ipw.HBox([fig, html])
13 except:
14     box = '### Install `plotly` to view output'
```



Writing Plotly Figure

Install plotly to view output

Python

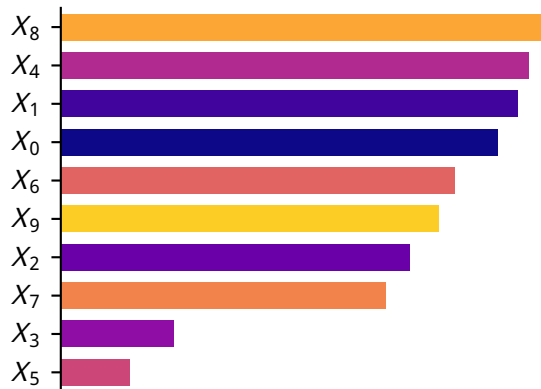
```
1 try:
2     import ipywidgets as ipw
3     import plotly.graph_objects as go
4     fig = slides.patched_plotly(go.FigureWidget()) # prefere Widget for interactivity and correct display
5     fig.add_trace(go.Bar(y=[1,5,8,9], customdata=["A","B"]))
6
7     # We have clicked and selected traits on patched plotly
8     html = ipw.HTML()
9     def observe_click(change):
10         html.value = "<br/>".join(f" {k} = {v}" for k, v in change['new'].items())
11     fig.observe(observe_click, names='clicked')
12     box = ipw.HBox([fig, html])
13 except:
14     box = '### Install `plotly` to view output'
```



Refreshable Content

Use refresh button below to update plot! See `race_plot` function at end of slides.

Race Plot



A Silly Plot

 Refresh Plot

Python

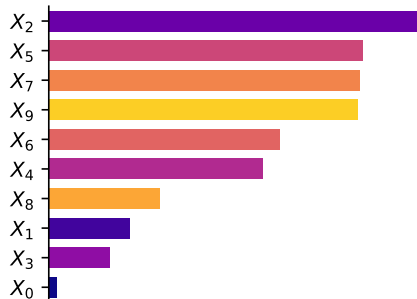
```
1 write('''
2     ## Refreshable Content
3     Use refresh button below to update plot!
4     See [alert! race_plot /] function at end of slide
5     ''')
6
7 def display_plot(btn): return race_plot().display()
8
9 write(
10     slides.dl.interactive(display_plot, btn = slides.get_source()
11     rslide.get_source()
12 ) # Only first columns will update
```



Animations with Widgets

anim    0 Animation Frame: 0

Race Plot



A Silly Plot

Python

```
1 write('## Animations with Widgets')
2 anim = slides.AnimationSlider(nframes=20, interval=1500, c
3 css = {'grid-template-columns': '1fr 2fr', '.out-main': {
4
5 @slides.dl.interact(post_init = lambda self: self.set_css(
6 def _(html, source, anim):
7     html.value = race_plot().value
8     print(f'Animation Frame: {anim}') # goes to output are
```



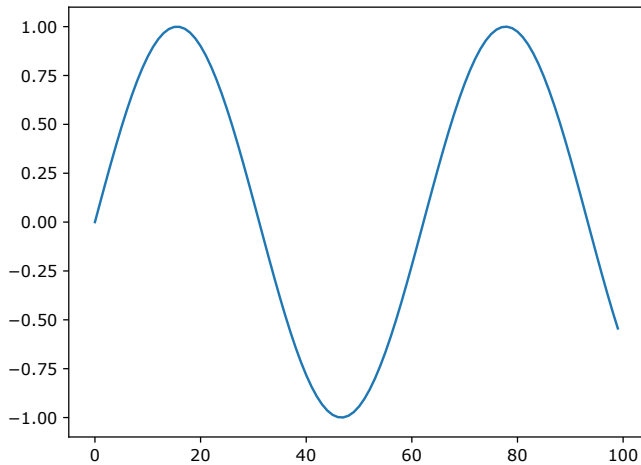
Rich Content ListWidget

Execute a code block

```
lambda: print(np.random.random((10,2))),
```

```
lambda: plt.plot(np.random.random((10,2))),
```

```
def plot_sine():
    plt.plot(np.sin(np.linspace(0,10,100)))
```



Python

```

1 @slides.src
2 def _(s):
3     import numpy as np
4     import matplotlib.pyplot as plt
5     def plot_sine():
6         plt.plot(np.sin(np.linspace(0,10,100)))
7     lw = slides.ListWidget(description='Execute a code block',
8                             options = [
9         lambda: print(np.random.random((10,2))),
10        lambda: plt.plot(np.random.random((10,2))),
11        plot_sine,
12    ], transform = lambda value: slides.code(value))
13
14 def run(c):
15     if callable(c): c() # avoid None value when
16     plt.show()
17
18 css = {'out-main': {'height':'300px'}, 'grid':
19
20 it = slides.dl.interactive(run, c = lw, post_in
21 write(['### Rich Content [code! ListWidget /]',

```



Contents

1. Introduction
2. Adding informative TOC
3. Plotting and DataFrame
- 4. Simple Animations with Frames**
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides

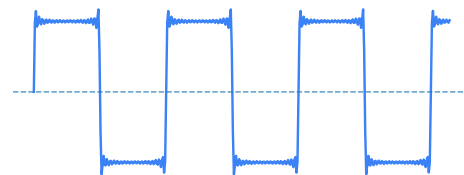


→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 plots = [] # row list for in place reveal
4 x = np.linspace(0, 2 * np.pi, 400)
5 for idx in range(1,20):
6     fig, ax = plt.subplots(figsize=(3.4, 2))
7     y = np.zeros_like(x)
8     for n in range(1, idx + 1):
9         k = 2 * n - 1
10        y += (1 / k) * np.sin(np.pi * k * x)
11    y *= (4 / np.pi)
12
13    ax.plot(x, y, color='#3b82f6', lw=1)
14    ax.axhline(0, alpha=0.75, lw=0.6, ls='dashed')
15    ax.set_ylim(-1.9, 1.9)
16    ax.axis('off')
17    fig.patch.set_facecolor('#111827')
18    plots.append(slides.plt2html(fig, caption=f"Harmonics: {idx}"))
```

Animating Matplotlib!



Harmonics: 19

6 ●

`steps(plots, 'bottom', interval=800)`

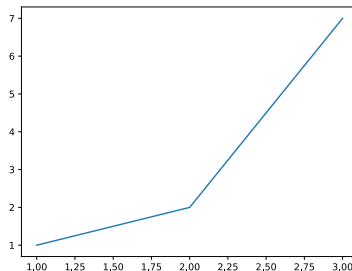


Note

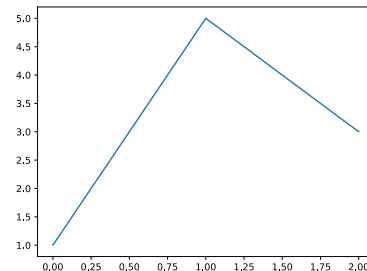
Only one frame (default last) is exported to PDF/HTML.



Inline Python Functions



Matplotlib in Markdown!



Default Plot

Python

```

1 @xmd.register("plot")
2 @wraps(plt.plot)
3 def _(*args, kind="plot", **kwargs):
4     caption = kwargs.pop("caption", None) #
5     getattr(plt, kind)(*args, **kwargs) # h
6     return slides.plt2html(caption = caption)

```

Markdown

```

1 ::: columns.inline 3 2 2
2     ## Inline Python Functions
3     [plot!! [1,2,3], [1,2,7], caption="Matplotlib [aler
4     [plot!! [1,5,3], caption="Default Plot" /]
5
6 ::: columns.inline .. {%sc} | [md-src/]

```



Contents

1. Introduction
2. Adding informative TOC
3. Plotting and DataFrame
4. Simple Animations with Frames
- 5. Controlling Content on Frames**
6. Custom Objects Serilaization
7. Code to Generate Slides



Frames with Incremental Rows

Python

```
1 boxes = [slides.html('div',
2     [slides.badge(str(c), f'rgb({c/3*255}, 0, {(3-c)/3*255})', round=True), f' :: block .. **{c*10}**'],
3     css_class="columns inline") for c in range(4)]
4 write('# Frames with Incremental Rows [yoffset! 0 /]')
5 s.get_source().focus([0,1,2,5]).display()
6 slides.pause() # split write from previous content
7 write(boxes, css_class='anim-group anim-slide-left', paused=True) # rows reveal one by one
```



Frames with Incremental Rows

Python

```
1 boxes = [slides.html('div',
2     [slides.badge(str(c), f'rgb({c/3*255}, 0, {(3-c)/3*255})', round=True), f' :: block .. **{c*10}**'],
3     css_class="columns inline") for c in range(4)]
4 write('# Frames with Incremental Rows [yoffset! 0 /]')
5 s.get_source().focus([0,1,2,5]).display()
6 slides.pause() # split write from previous content
7 write(boxes, css_class='anim-group anim-slide-left', paused=True) # rows reveal one by one
```

0

0



Frames with Incremental Rows

Python

```
1 boxes = [slides.html('div',
2     [slides.badge(str(c), f'rgb({c/3*255}, 0, {(3-c)/3*255})', round=True), f' :: block .. **{c*10}**'],
3     css_class="columns inline") for c in range(4)]
4 write('# Frames with Incremental Rows [yoffset! 0 /]')
5 s.get_source().focus([0,1,2,5]).display()
6 slides.pause() # split write from previous content
7 write(boxes, css_class='anim-group anim-slide-left', paused=True) # rows reveal one by one
```

0 0

1 10



Frames with Incremental Rows

Python

```
1 boxes = [slides.html('div',
2     [slides.badge(str(c), f'rgb({c/3*255}, 0, {(3-c)/3*255})', round=True), f' :: block .. **{c*10}**'],
3     css_class="columns inline") for c in range(4)]
4 write('# Frames with Incremental Rows [yoffset! 0 /]')
5 s.get_source().focus([0,1,2,5]).display()
6 slides.pause() # split write from previous content
7 write(boxes, css_class='anim-group anim-slide-left', paused=True) # rows reveal one by one
```

0 0

1 10

2 20



Frames with Incremental Rows

Python

```
1 boxes = [slides.html('div',
2     [slides.badge(str(c), f'rgb({c/3*255}, 0, {(3-c)/3*255})', round=True), f' :: block .. **{c*10}**'],
3     css_class="columns inline") for c in range(4)]
4 write('# Frames with Incremental Rows [yoffset! 0 /]')
5 s.get_source().focus([0,1,2,5]).display()
6 slides.pause() # split write from previous content
7 write(boxes, css_class='anim-group anim-slide-left', paused=True) # rows reveal one by one
```

0 0

1 10

2 20

3 30



Frames with

2x2 grid of boxes

Python

```
1 write('# Frames with \n#### 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 slides.pause() # split from previous content
6 for ws, cols in zip(widths,objs): # isolate to split from previous content
7     write(*cols, widths=ws, css_class='anim-group anim-wipe-right', paused=True) # incremental row reveal
```



Frames with

2x2 grid of boxes

Python

```
1 write('# Frames with \n#### 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 slides.pause() # split from previous content
6 for ws, cols in zip(widths,objs): # isolate to split from previous content
7     write(*cols, widths=ws, css_class='anim-group anim-wipe-right', paused=True) # incremental row reveal
```

0

0



Frames with

2x2 grid of boxes

Python

```
1 write('# Frames with \n#### 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 slides.pause() # split from previous content
6 for ws, cols in zip(widths,objs): # isolate to split from previous content
7     write(*cols, widths=ws, css_class='anim-group anim-wipe-right', paused=True) # incremental row reveal
```

0 0

1 10



Frames with

2x2 grid of boxes

Python

```
1 write('# Frames with \n#### 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 slides.pause() # split from previous content
6 for ws, cols in zip(widths,objs): # isolate to split from previous content
7     write(*cols, widths=ws, css_class='anim-group anim-wipe-right', paused=True) # incremental row reveal
```

0 0 1 10

2 20

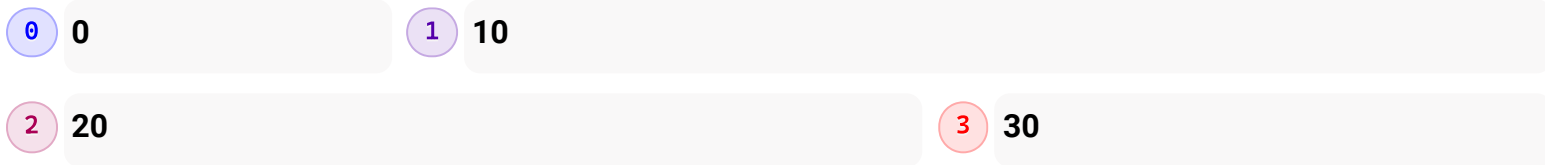


Frames with

2x2 grid of boxes

Python

```
1 write('# Frames with \n#### 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 slides.pause() # split from previous content
6 for ws, cols in zip(widths,objs): # isolate to split from previous content
7     write(*cols, widths=ws, css_class='anim-group anim-wipe-right', paused=True) # incremental row reveal
```





← Skip Previous Frames

Watching Youtube Video?

Want to do some drawing instead? Click on button on the right!



IPySlides-Demo
Abdul Saboor



Watch on



IPySlides | From a Hobby to a Sc
Abdul Saboor



Watch on

✓ Show Code



Blocks with CSS classes

Table

h1	h2	h3
d1	d2	d3
r1	r2	r3

Widgets



Click to do nothing

☐ Select to do nothing

A rich content table

h1	h2	h3
↪	2	3
3	<code>import</code> numpy <code>as</code> np	5

✓ Show Code



$LAT_E X$ in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.

💡 LaTeX Tip

Variable formatting alongwith $LAT_E X$ `%{var}` → 'I was a variable' is seamless.



$LAT_E X$ in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.

$$\int_0^1 \frac{1}{1-x^2} dx$$

💡 LaTeX Tip

Variable formatting alongwith $LAT_E X$ `%{var}` → 'I was a variable' is seamless.



$LAT_E X$ in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.

💡 LaTeX Tip

Variable formatting alongwith $LAT_E X$ `%{var}` → 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

$$ax^2 + bx + c = 0$$



LAT_{EX} in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations LAT_{EX} needs time to load, so keeping it in view until it loads would help.

💡 LaTeX Tip

Variable formatting alongwith LAT_{EX} `%{var}` → 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

Hello 🙋 I am pinned here!
 $ax^2 + bx + c = 0$



LAT_{EX} in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations LAT_{EX} needs time to load, so keeping it in view until it loads would help.

💡 LaTeX Tip

Variable formatting alongwith LAT_{EX} `%{var}` → 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

Hello 🙋 I am pinned here!
 $ax^2 + bx + c = 0$

✓ Show Code



Serialize Custom Objects to HTML

This is useful for displaying user defined/third party objects in slides

0 1 2 3 4 5 6 7 8 9

Python

```
1 write('## Serialize Custom Objects to HTML\nThis is useful for displaying user defined/third party objects in
2 with slides.suppress_stdout(): # suppress stdout from register fuction below
3     @slides.serializer.register(int)
4     def colorize(obj):
5         color = 'red' if obj % 2 == 0 else 'green'
6         return f'<span style="color:{color};">{obj}</span>'
7     write(*range(10))
8
9 some_slide.get_source().display()
```



Source Code

Markdown: Slide 0

```
1  ```md-src.collapsed
2  # Creating Slides
3  ::: align-center width=50%
4      [alert! Abdul Saboor /][sup!1/], Unknown Author[sup!2/]
5      [center! [today!/] /]
6      ::: align-left color=var(--fg2-color)
7          [sup!1/]My University is somewhere in the middle of nowhere
8          [sup!2/]Their University is somewhere in the middle of nowhere
9
10 [gap!! v=2 /]
11 ::: note.ask block-clear head="Looking for Slides Controls?" width=90cqW
12     - Right-click (or click the footer) to open the context menu for settings and contents.
13     - Hover top-left and click [fa! caret-right /] to open the menu when the footer or right-click :
14     - Click a footer section (when shown) to jump there quickly.
15 ```
16 [md-src/]
```

Markdown: Slide 1

```
1 [section! Introduction /] [toc! ### Contents /]
```

Markdown: Slide 2

```
1 # Introduction
```

```
2
```